AGENTS.md: Observability Console

Read `SPEC.md` first; it is the source of truth. This file explains how to work in the repo.

This is a fully in-house product. Don't introduce Grafana, Metabase, Superset, Kibana or any other off-the-shelf dashboard tool. We build the UI, the API, the collectors and the alerting ourselves.

Priorities

1. **Demo mode must always work offline.** `make demo` should serve the console at http://localhost:8080 with seeded, live-updating data and no external network calls. Never break this.
2. **Build milestones in order (SPEC §17).** After M4, stop and summarize what you built, what you assumed, and what's left.
3. **Never call production systems.** Live mode connects only through env vars a human sets. Tests use recorded JSON fixtures.

Backend conventions (Python 3.11)

- Libraries: FastAPI, psycopg 3 (plain SQL in small repository modules, no ORM), pydantic and pydantic-settings, httpx, APScheduler, sse-starlette, PyYAML, passlib[bcrypt] or argon2, PyJWT.
- Type hints everywhere. Use ruff for lint and format, and pytest for tests.
- Migrations are numbered SQL files in `sql/migrations/`, applied by `python -m obs.migrate`. Never edit an applied migration; add a new one.
- All collector writes are upserts on natural keys, so every process is safe to restart.
- Store `timestamptz` in UTC, and return ISO-8601 UTC from the API.
- All error text and logs go through `redact.py` before they are stored and before they are returned.
- One failing tenant or API call must never crash a loop. Record it in `collector_state.last_error` or `platform_health` and continue.
- Every mutating endpoint checks the role and writes `audit_log`.

Frontend conventions

- React + TypeScript (strict), Vite, Tailwind, TanStack Query and Table, Apache ECharts, React Router.
- API types are generated from OpenAPI (`make types`). Never hand-write API response types.
- Status colors and labels live in one place (`components/StatusPill.tsx` plus a shared color map). Use the same colors on every page.
- Every page supports light and dark mode, shareable URL state (filters and range in the query string), loading skeletons, and a useful empty state.
- Live updates go through `useLiveEvents` (SSE), which invalidates TanStack Query caches. Keep 30-second polling as a fallback.
- Use ESLint and Prettier, Vitest for component tests, and Playwright for e2e.

Commands

Command	What it does
`make up`	Start the stack in the current MODE
`make demo`	MODE=demo, start the stack, seed data, ticker on
`make live`	MODE=live using `.env`
`make down`	Stop the stack
`make reset`	Stop the stack and drop volumes
`make seed`	Re-run the demo seed
`make types`	Regenerate the frontend TS types from the API's OpenAPI
`make test`	Backend pytest plus frontend Vitest
`make e2e`	Playwright smoke test against a running demo
`make lint`	ruff, ESLint, Prettier check, tsc

Definition of done (every change)

- `make lint` , `make test` and `make e2e` pass.
- `make reset && make demo` works from scratch.

- Every SPEC §16 scenario is visible in the UI. No page shows an error or an unexpected empty state in demo mode.
- The Playwright smoke test covers: log in → overview renders → click the red tenant → open the failed run → log is visible → retry → the new run appears.
- New env vars are added to `.env.example` and mentioned in SPEC.
- New error patterns come with a test in `test_classify.py`.

Don't

- Don't add Prometheus, Loki, Kafka, Redis or any other infrastructure in phase 1. Postgres `LISTEN` / `NOTIFY` covers live events.
- Don't let the browser talk to Airflow, Airbyte or the warehouse directly; everything goes through `/api`.
- Don't store raw passwords from the onboarding wizard. Store secret references only.
- Don't store free-form SQL in alert rules. Rule types are code; only parameters are editable.
- Don't run `count(*)` on warehouse tables in live mode. Use catalog statistics.
- Don't modify the team's existing DAGs from this repo. Ship `airflow/dags/README.md` with the migration steps instead.